

Justificacion del uso de SQL (PostgreSQL / Supabase)

Trabajo Practico - LinkPro (clon de LinkedIn)

1. Contexto del trabajo practico

El TP consiste en construir una red social similar a LinkedIn integrando multiples motores de bases de datos para distintas funcionalidades. La consigna exige usar al menos una base de datos relacional (SQL), una documental (MongoDB) y una de grafos o clave-valor. La eleccion de que tecnologia usar para cada modulo debe estar justificada por las características propias de los datos y las operaciones que se realizan sobre ellos.

2. Funcionalidades implementadas sobre SQL

Las siguientes funcionalidades del sistema se apoyan en tablas SQL gestionadas a traves de Supabase (PostgreSQL):

Modulo	Tablas involucradas
Usuarios y autenticacion	users, roles, user_roles
Perfil profesional	users, work experience, education, skills, user_skill
Empresas	companies
Ofertas de empleo	jobs, job_skill
Postulaciones	applications

3. Por que SQL para estas funcionalidades

3.1 Integridad referencial y relaciones fuertes

Los datos de usuarios, perfiles, empleos y postulaciones estan fuertemente relacionados entre si. Las claves foraneas de SQL garantizan que no existan postulaciones huérfanas, ni ofertas que referencien usuarios o empresas inexistentes. Esto seria mucho mas dificil de garantizar en una base de datos documental sin logica de aplicacion adicional.

- Una oferta de empleo (jobs) pertenece a un usuario (poster_user_id -> users.user_id) y opcionalmente a una empresa (company_id -> companies).
- Una postulacion (applications) vincula un usuario con una oferta (user_id -> users, job_id -> jobs).
- Las habilidades del usuario (user_skill) y las requeridas por una oferta (job_skill) referencian el catalogo comun skills.

3.2 Consultas relacionales complejas en una sola operacion

La API utiliza las capacidades de join de Supabase PostgREST para traer datos de multiples tablas en una sola consulta. Por ejemplo, al listar ofertas:

```
jobs
+-- companies(name)
+-- job_skill
```

```
+-- skills(skill_id, name, type)
```

Y al listar postulaciones de un usuario:

```
applications
+-- jobs
|   +-- companies(name)
+-- users(user_id, name, surname, email, profile_photo_url)
```

Con documentos MongoDB habria que hacer multiples consultas o disenar documentos embebidos que no representan bien la realidad del modelo, ya que las mismas skills son compartidas por usuarios y ofertas, y las mismas companies son compartidas por experiencias laborales y ofertas.

3.3 Normalizacion: evitar duplicacion de datos

El catalogo de habilidades (skills) es un claro ejemplo de por que la normalizacion relacional es conveniente:

- Una habilidad como "Python" existe una sola vez en la tabla skills.
- Tanto el perfil de un usuario (user_skill) como una oferta de empleo (job_skill) la referencian mediante su skill_id.
- Si el nombre de una habilidad cambiara, se actualiza en un unico lugar.

Con un modelo documental, "Python" apareceria embebida en el documento de cada usuario y en el de cada oferta, generando inconsistencias.

3.4 Unicidad y restricciones de negocio

SQL permite declarar restricciones directamente en el esquema, garantizandolas en la capa de almacenamiento:

- users.email es UNIQUE: no pueden existir dos cuentas con el mismo email.
- users.dni es UNIQUE: no puede haber duplicados de documento.
- applications(job_id, user_id) es unico: un usuario no puede postularse dos veces a la misma oferta.
- Campos como is_active, status e is_current tienen valores por defecto y restricciones de tipo.

3.5 Transacciones ACID

Las operaciones criticas del sistema requieren atomicidad:

- Al registrar un usuario: se inserta en users y se asignan roles en user_roles. Si una operacion falla, la cuenta no queda en estado inconsistente.
- Al publicar una oferta con habilidades: se inserta en jobs y luego en job_skill de forma atomica.

Las garantias ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) de PostgreSQL aseguran consistencia incluso bajo fallas parciales o concurrencia de multiples usuarios.

3.6 Esquema predecible para datos estructurados

Los datos de usuarios, perfiles y empleos tienen una estructura fija y bien definida: todos los usuarios tienen nombre, email y contrasena; todas las ofertas tienen titulo, modalidad y estado. Esta homogeneidad favorece el modelo relacional. Un motor documental como MongoDB es mas conveniente cuando los datos son heterogeneos o evolucionan rapidamente en estructura (por ejemplo, el contenido de posts o comentarios).

4. Queries ejecutadas desde el backend

El backend (Flask) dispara las queries SQL automaticamente al iniciar, creando las tablas si no existen (via init_db.py con psycopg2). Durante la ejecucion, cada ruta accede a Supabase usando el SDK PostgREST, que construye y ejecuta las queries SQL internamente. A continuacion se detallan las operaciones por modulo:

4.1 Modulo de autentificacion (auth.py)

REGISTRO - Inserta usuario y asigna rol. Si el rol es 'poster', busca o crea la empresa:

```
INSERT INTO users (user_id, email, password_hash, name, surname, dni)
VALUES ($1, $2, $3, $4, $5, $6)
```

```
SELECT company_id FROM companies WHERE name ILIKE $1 LIMIT 1
```

```
-- Si no existe:
```

```
INSERT INTO companies (company_id, name) VALUES ($1, $2)
```

```
INSERT INTO user_roles (user_id, role_id) VALUES ($1, $2)
```

LOGIN - Busca el usuario por email y verifica la contraseña con bcrypt:

```
SELECT user_id, email, name, surname, dni, profile_photo_url, password_hash
FROM users WHERE email = $1 LIMIT 1
```

```
SELECT roles.name FROM user_roles
JOIN roles ON user_roles.role_id = roles.role_id
WHERE user_roles.user_id = $1
```

CAMBIAR CONTRASEÑA (reset):

```
UPDATE users SET password_hash = $1 WHERE user_id = $2
-- Nota: el token de reset se guarda en memoria del servidor (dict Python),
-- no en la base de datos, para simplificar el entorno de desarrollo.
```

4.2 Modulo de perfil (profile.py)

ACTUALIZAR DATOS PERSONALES:

```
UPDATE users SET name=$1, surname=$2, dni=$3, updated_at=now()
WHERE user_id = $4
```

```
-- Foto: se guarda en backend/static/photos/ y se actualiza la URL:
```

```
UPDATE users SET profile_photo_url=$1 WHERE user_id = $2
```

EXPERIENCIA LABORAL (work experience):

```
-- Listar:
```

```
SELECT we.*, companies.name FROM "work experience" we
LEFT JOIN companies ON we.company_id = companies.company_id
WHERE we.user_id = $1 ORDER BY from_date DESC
```

```
-- Agregar:
```

```
INSERT INTO "work experience"
```

```
(user_id, company_id, title, description, from_date, end_date, is_current)
VALUES ($1, $2, $3, $4, $5, $6, $7)
```

-- Eliminar:

```
DELETE FROM "work experience" WHERE we_id=$1 AND user_id=$2
```

EDUCACION:

-- Listar:

```
SELECT * FROM education WHERE user_id=$1 ORDER BY from_date DESC
```

-- Agregar:

```
INSERT INTO education
  (user_id, title, field, institution, from_date, end_date, is_actual)
VALUES ($1, $2, $3, $4, $5, $6, $7)
```

-- Eliminar:

```
DELETE FROM education WHERE edu_id=$1 AND user_id=$2
```

HABILIDADES (skills / user_skill):

-- Buscar o crear skill en el catalogo:

```
SELECT skill_id FROM skills WHERE name ILIKE $1 LIMIT 1
INSERT INTO skills (name, type) VALUES ($1, $2)
```

-- Agregar o actualizar nivel de una habilidad en el perfil:

```
INSERT INTO user_skill (user_id, skill_id, level) VALUES ($1, $2, $3)
ON CONFLICT (user_id, skill_id) DO UPDATE SET level=$3
```

-- Listar habilidades del perfil:

```
SELECT us.level, s.skill_id, s.name, s.type
  FROM user_skill us JOIN skills s ON us.skill_id = s.skill_id
 WHERE us.user_id = $1
```

-- Eliminar:

```
DELETE FROM user_skill WHERE user_id=$1 AND skill_id=$2
```

4.3 Modulo de empleos (jobs.py)

LISTAR OFERTAS ACTIVAS - JOIN con companies y skills en una sola operacion:

-- El SDK PostgREST ejecuta internamente:

```
SELECT jobs.*, companies.name,
       job_skill.skill_id, skills.name, skills.type
  FROM jobs
 LEFT JOIN companies ON jobs.company_id = companies.company_id
 LEFT JOIN job_skill  ON job_skill.job_id = jobs.job_id
 LEFT JOIN skills     ON skills.skill_id  = job_skill.skill_id
 WHERE jobs.is_active = TRUE
```

```
ORDER BY jobs.created_at DESC
```

CREAR OFERTA Y AGREGAR SKILLS:

```
INSERT INTO jobs
  (job_id, poster_user_id, company_id, title, description,
   location, shift, modality, is_active)
VALUES ($1,$2,$3,$4,$5,$6,$7,$8,TRUE)
```

-- Para cada skill requerida:

```
INSERT INTO job_skill (job_id, skill_id) VALUES ($1, $2)
```

ELIMINAR OFERTA - ON DELETE CASCADE elimina automaticamente job_skill y applications:

```
DELETE FROM jobs WHERE job_id=$1
```

-- Cascade: borra automaticamente job_skill y applications asociadas

4.4 Modulo de postulaciones (jobs.py)

POSTULARSE A UNA OFERTA:

```
INSERT INTO applications (application_id, job_id, user_id, status)
  VALUES ($1, $2, $3, 'submitted')
-- UNIQUE (job_id, user_id) impide postulaciones duplicadas
```

VER MIS POSTULACIONES (candidato) - JOIN de 3 tablas:

```
SELECT applications.*, jobs.job_id, jobs.title,
       companies.name, users.user_id, users.name,
       users.surname, users.email
FROM applications
LEFT JOIN jobs      ON applications.job_id = jobs.job_id
LEFT JOIN companies ON jobs.company_id     = companies.company_id
LEFT JOIN users     ON applications.user_id = users.user_id
WHERE applications.user_id = $1
ORDER BY applications.applied_at DESC
```

VER POSTULANTES DE UNA OFERTA (poster) Y CAMBIAR ESTADO:

```
SELECT applications.*, users.name, users.surname,
       users.email, users.profile_photo_url
FROM applications
JOIN users ON applications.user_id = users.user_id
WHERE applications.job_id = $1

UPDATE applications SET status=$1, updated_at=now()
WHERE application_id=$2
```

5. Diagrama de relaciones (simplificado)

users ----- user_roles ---- roles

```
|
+--- work experience ---- companies
+--- education
+--- user_skill ----- skills
|
+--- jobs (poster_user_id) --- companies
|
+--- job_skill -----++
|
+--- applications ---- users
```

6. Integracion en la arquitectura del sistema

Backend: Flask + Supabase Python SDK

El backend expone una API REST construida con Flask, organizada en blueprints por dominio:

- /auth -> auth_bp: registro, login, recuperacion de contraseña.
- /profile -> profile_bp: datos del perfil, foto, experiencia, educacion, habilidades.
- /jobs -> jobs_bp: listado de ofertas, publicacion, postulaciones, gestion de candidatos.

Cada blueprint interactua con Supabase mediante el cliente supabase_admin (clave de servicio), que ejecuta queries sobre PostgreSQL usando la API PostgREST.

Supabase como infraestructura SQL

- PostgreSQL gestionado: base de datos relacional con todas las garantias de consistencia.
- PostgREST: convierte automaticamente las tablas en endpoints REST con soporte de filtros, joins y paginacion.
- Fotos de perfil: se almacenan en el sistema de archivos del servidor Flask (static/photos/) y se sirven como archivos estaticos desde <http://localhost:5000/static/photos/>. La URL resultante se guarda en el campo profile_photo_url de la tabla users.
- Row Level Security (RLS): politicas de acceso a nivel de fila. En el TP se usa la clave service_role desde el backend.
- Tokens de recuperacion de contraseña: se almacenan en memoria del servidor (diccionario Python) durante la sesion activa, con expiracion de 1 hora. Este enfoque simplifica el desarrollo evitando una tabla de tokens en la BD.

7. Comparacion con las otras bases del proyecto

Criterio	SQL (PostgreSQL)	MongoDB (documental)	Grafo / Clave-valor
Estructura	Fija, tabular	Flexible, anidada	Relaciones / pares
Relaciones	Claves foraneas, JOINS	Referencias manuales	Aristas del grafo
Caso de uso TP	Usuarios, perfiles, empleos	Posts, comentarios, grupos	Conexiones entre usuarios
Justificacion	Datos con muchas relaciones	Contenido heterogeneo	Traversals de red social

8. Conclusion

La eleccion de SQL para los modulos de usuarios, perfiles, empleos y postulaciones esta justificada por:

- La naturaleza relacional de los datos: usuarios, empresas, ofertas y postulaciones se interconectan mediante referencias que deben ser consistentes.
- La necesidad de integridad referencial: las claves foraneas garantizan que los datos no queden en estados invalidos.
- La eficiencia de las consultas con JOINS: se obtienen datos de multiples tablas en una unica operacion.
- La normalizacion: skills y companies son entidades compartidas que no deben duplicarse.
- Las restricciones de unicidad: email, DNI y postulaciones unicas por usuario/oferta son propiedades del esquema.
- Las garantias ACID: operaciones criticas como el registro de usuarios o la publicacion de ofertas son atomicas.

Estas caracteristicas hacen de PostgreSQL/Supabase la tecnologia adecuada para el nucleo transaccional del sistema, mientras que MongoDB y otras bases complementan el sistema para datos menos estructurados como publicaciones, comentarios y conexiones sociales.